SHERLOCK

Security Review For Starknet



Collaborative Audit Prepared For:	**Starknet**
Lead Security Expert(s):	**bin2chen** **EV_om**
Date Audited:	**June 9 - June 16, 2025**
Final Commit:	**0eaa3bc**

Introduction

StarkWare, the company behind Starknet, is integrating Lombard Protocol's LBTC, a liquid staking token for Bitcoin, into the Starknet ecosystem. This integration aims to bring Bitcoin's liquidity and DeFi opportunities to Starknet, a leading ZK Rollup. The collaboration will allow users to stake native Bitcoin for LBTC and seamlessly bridge it to and from Starknet.

Scope

Repository: starkware-libs/sn-lbtc

Audited Commit: 6024f37cd13512a0c583016d3185d28726b70ad9

Final Commit: 0eaa3bcf17c60033bf6eb759ffd47a4be2656032

Files:

- packages/bascule/README.md
- packages/bascule/Scarb.toml
- packages/bascule/src/bascule.cairo
- packages/bascule/src/errors.cairo
- packages/bascule/src/events.cairo
- packages/bascule/src/interface.cairo
- packages/bascule/src/lib.cairo
- packages/bridge/README.md
- packages/bridge/Scarb.toml
- packages/bridge/src/bitcoin_utils.cairo
- packages/bridge/src/bridge.cairo
- packages/bridge/src/errors.cairo
- packages/bridge/src/events.cairo
- packages/bridge/src/interface.cairo
- packages/bridge/src/lib.cairo
- packages/bridge/src/utils.cairo
- packages/consortium/README.md
- packages/consortium/Scarb.toml
- packages/consortium/src/consortium.cairo
- packages/consortium/src/errors.cairo

- packages/consortium/src/events.cairo
- packages/consortium/src/interface.cairo
- packages/consortium/src/lib.cairo
- packages/consortium/src/utils.cairo
- packages/erc4626/src/erc4626.cairo
- packages/erc4626/src/interface.cairo
- packages/erc4626/src/lib.cairo
- packages/erc4626/src/math.cairo
- packages/manual_strategy/README.md
- packages/manual_strategy/Scarb.toml
- packages/manual_strategy/src/errors.cairo
- packages/manual_strategy/src/events.cairo
- packages/manual_strategy/src/interface.cairo
- packages/manual_strategy/src/lib.cairo
- packages/manual_strategy/src/manual_strategy.cairo
- packages/stake_and_bake/src/errors.cairo
- packages/stake_and_bake/src/events.cairo
- packages/stake_and_bake/src/interface.cairo
- packages/stake_and_bake/src/lib.cairo
- packages/stake_and_bake/src/stake_and_bake.cairo
- packages/token/README.md
- packages/token/Scarb.toml
- packages/token/src/lbtc.cairo
- packages/token/src/lbtc/events.cairo
- packages/token/src/lbtc/token.cairo
- packages/token/src/lib.cairo
- packages/vault/README.md
- packages/vault/Scarb.toml
- packages/vault/src/errors.cairo
- packages/vault/src/events.cairo
- packages/vault/src/interface.cairo

- packages/vault/src/lib.cairo
- packages/vault/src/vault.cairo

Final Commit Hash

0eaa3bcf17c60033bf6eb759ffd47a4be2656032

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	5	15

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Incorrect `SINGLE_ASSET` constant causes excessive performance fees

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/60

Summary

The `SINGLE_ASSET` constant is set to 1e18 instead of 1e17, causing performance fee calculations to be overinflated and resulting in massive user dilution.

Vulnerability Detail

The SINGLE_ASSET constant in `vault.cairo` is incorrectly defined as `10_u256.pow(18)` (1e18), when it should be `10_u256.pow(17)` (1e17) based on the vault's decimal configuration. The vault uses an underlying asset with 8 decimals and applies a 9-decimal offset, as defined in `ImmutableConfigImpl`.

In accrued_performance_fee(), this constant is used to calculate the current share value. When the vault realizes a 1% profit, the calculation should yield:

- Correct: share_value = 1_010_000_000, difference from high water mark = 10_000_000
- Incorrect: share_value = 10_100_000_000 (10x inflated), difference = 9_100_000_000

This creates a 910x multiplier (`9_100_000_000 / 10_000_000 = 910`) in the performance fee calculation on first run, since the fee is proportional to `(share_value - high_water_mark) * total_assets`.

Impact

Users face catastrophic dilution from performance fees. With a 1% vault profit and 10% performance fee rate, users should experience 0.1% dilution (10% of 1% profit). Instead, they face 91% dilution - losing half their holdings.

Code Snippet

Tool Used

Manual Review

Recommendation

Correct the decimal precision to match the vault's 17-decimal configuration:

```
- const SINGLE_ASSET: u256 = 10_u256.pow(18);
+ const SINGLE_ASSET: u256 = 10_u256.pow(17);
```

Issue M-1: btc_locked_amount should be reduced by amount_after_fee

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/51

Summary

When `redeem()`, btc_locked_amount decreases by `amount`, but lbtc actually `burns` by amount_after_fee

Vulnerability Detail

The `redeem()` code is as follows.

currently using `btc_locked_amount -= amount`. correctly should reduce `btc_locked_amount -= amount_after_fee`. since lbtc `burn(amount)` first, then `mint(fee)`, so the actual amount `burn` is `amount_after_fee`

Impact

`btc_locked_amount` is too small and some of the `lbtc` will not be redeemed.

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/bridge/src/bridge.cairo#L180

Recommendation

```
            // Validate there are enough locked btc on the other side
            let locked_amount = self.btc_locked_amount.read();
            assert(locked_amount >= amount, errors::REDEEM_LIMIT_EXCEEDED);
-           self.btc_locked_amount.write(locked_amount - amount);
+           self.btc_locked_amount.write(locked_amount - amount_after_fee);
```

Discussion

bin2chen66

Fixed change to `self.btc_locked_amount -= amount_after_fee`

Issue M-2: when vault.deposit() ,incorrect calculation of fee_shares

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/54

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

incorrect calculation of fee_shares

Vulnerability Detail

when deposit(), We mint fee through `after_deposit()`

current formula: fee_shares = `fee_assets * total_supply / total_assets`

but at this point, total_supply has been increased, but total_assets has not been increased, because total_assets is increased after `strategy_address.deposit()` is performed later in this method.

example: total_supply = 100 total_assets = 100 exchange rate = 1:1 fees = 0.04

fee_shares = fee_assets * total_supply / total_assets = 4 * (100 + 96) / 100 = 7.84

Correctly it should be 4.

Impact

Overcharging fees

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/vault/src/vault.cairo#L327

Recommendation

```
        fn after_deposit(
            ref self: ERC4626ComponentState<ContractState>, assets: u256, shares:
             ↪  u256,
        ) {
            let mut vault = self.get_contract_mut();
```

```
            // Check DUST limit.
            assert(shares > DUST, SHARES_BELOW_DUST);

            let fee_recipient = vault.get_fee_recipient();
            if get_caller_address() != fee_recipient {
                // Calculate again the fee in shares.
                let fee_assets = u256_mul_div(
                    x: assets,
                    y: vault.get_deposit_fee().into(),
                    denominator: MAX_BASIS_POINTS.into(),
                    rounding: Rounding::Ceil,
                );
-               let fee_shares = self
-                   ._convert_to_shares(assets: fee_assets, rounding:
 ↪  Rounding::Ceil);
+               let fee_shares = u256_mul_div(
+                   fee_assets,
+                   vault.total_supply() - shares +
 ↪  10_u256.pow(ImmutableConfigImpl::DECIMALS_OFFSET.into()),
+                   self.total_assets() ,
+                   Rounding::Floor,
+               );

                if fee_shares.is_non_zero() {
                    vault.erc20.mint(recipient: fee_recipient, amount: fee_shares);
                }
            }

            let strategy_address = vault.strategy_address.read();
            IERC20Dispatcher { contract_address: self.ERC4626_asset.read() }
                .approve(spender: strategy_address, amount: assets);
            IERC4626Dispatcher { contract_address: strategy_address }
                .deposit(:assets, receiver: get_contract_address());
        }
    }
```

Discussion

MohammadNassar1

It was already fixed. New commit hash: 76124934507cb3bb7609ef70b2d497c7bc8ef9ba. working branch: dev. Thanks!

bin2chen66

Fixed in 76124934507cb3bb7609ef70b2d497c7bc8ef9ba

use computed fee_in_shares via `_preview_deposit()`

Issue M-3: performance fee incorrect calculation

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/55

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`accrued_performance_fee()` uses the wrong denominator when calculating fees

Vulnerability Detail

`accrued_performance_fee ()` For calculation of `performance fee`.

The code is as follows:

Currently using the formula: performance_fee = `(share_value - high_water_mark) * total_assets * fee / MAX_BASIS_POINTS / 18 ** UNDERLYING_DECIMALS`

`DECIMALS_OFFSET` should be used instead of `UNDERLYING_DECIMALS`.

because: high_water_mark = INIT_HIGH_WATER_MARK = 1e9 default exchange rate = 1e9:1

Impact

Charging excessive `performance fee`

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/vault/src/vault.cairo#L472

Recommendation

```
                        return u256_mul_div(
                            x: (share_value - high_water_mark) * total_assets,
                            y: fee.into(),
                            denominator: MAX_BASIS_POINTS.into()
-                               *
 ↪  10_u256.pow(ImmutableConfigImpl::UNDERLYING_DECIMALS.into()),
+                               *
 ↪  10_u256.pow(ImmutableConfigImpl::DECIMALS_OFFSET.into()),
```

Discussion

MohammadNassar1

Thanks for the suggestion. You're right that there is an issue with the performance calculation. But it's not related to the decimals. You're right, it seems that we have an issue with the current performance formula, but I don't think it's related to the decimals. The issue seems to be with the HighWaterMark, I think this parameter should align with the Underlying_decimals, which means the init_high_water_mark should be 10**8 and not 10**9. as it's compared against the ShareValue, which is 10**8.

bin2chen66

Fixed By modifying `INIT_HIGH_WATER_MARK = 1e8` , at the same time `DECIMALS_OFFSET=8` the number of bits has been aligned.

Issue M-4: Protocol fees round down to zero causing fee loss

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/63

Summary

The `accrued_protocol_fee()` function suffers from precision loss due to integer division, causing protocol fees to round down to zero for small time periods and reasonable TVL amounts.

Vulnerability Detail

The accrued_protocol_fee() function calculates protocol fees using the formula:

The issue lies in the denominator being significantly larger than the numerator for some scenarios. For example, with a 0.1% protocol fee (10 basis points), $1M TVL, and a 1-block time delta of approximately 30 seconds:

- `x` = 10 * 1e9 = 1e10
- `y` = 30
- `denominator` = ~3.15e11 (AVERAGE_YEAR * MAX_BASIS_POINTS)

This results in: (1e10 * 30) / 3.15e11 ≈ 0.95, which rounds down to 0 due to the `Rounding::Floor` parameter in `u256_mul_div()`.

Impact

The fee calculation guarantees revenue loss on every execution. Small vault TVL or brief time periods cause legitimate fees to round down to complete zero, while all calculations discard fractional amounts through `Rounding::Floor`, systematically undercharging users.

Code Snippet

Tool Used

Manual Review

Recommendation

Compute the fee in shares rather than assets to maintain precision, or implement a higher precision calculation by scaling the numerator appropriately before division. Consider accumulating fees over longer periods or using a different fee calculation mechanism that doesn't suffer from precision loss in short time intervals.

Discussion

0xEVom

@MohammadNassar1 @555-andrew this finding concerns the precision of the protocol fee and is not addressed by #250.

> with a 0.1% protocol fee (10 basis points), $1M TVL, and a 1-block time delta of approximately 30 seconds

That being said, I assume the protocol fee, as well as the TVL, can generally be expected to be significantly higher and this can be safely acknowledged.

Issue M-5: Performance fee tier selection prevents higher fee tiers from being reached

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/64

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The performance fee calculation uses an incorrect denominator when determining which fee tier to apply, preventing higher fee tiers from ever being reached due to frequent fee collection resetting the calculation baseline.

Vulnerability Detail

In the accrued_performance_fee() function, the fee tier selection is determined by calculating a `diff` value that represents the performance gain. However, the current implementation calculates this difference using `high_water_mark` as the denominator:

This approach measures performance gains only since the last time fees were collected, as `high_water_mark` gets updated each time `_take_fees()` is called. Since fees are collected frequently (on every deposit, mint, withdraw, redeem, and pause operation), the share value will only grow in small increments between fee collections. This means the `diff` value will always remain small and will never reach the higher performance fee tiers configured in the `performance_fee` vector.

The intended behaviour appears to be measuring performance against the initial baseline (`INIT_HIGH_WATER_MARK`) to determine the appropriate fee tier, while still only charging fees proportionally on the gains since the last collection (which is correctly implemented in the fee calculation portion using `share_value - high_water_mark`).

Impact

The protocol will never collect performance fees from higher fee tiers, regardless of how well the vault performs over time. This results in permanently reduced fee revenue for the protocol.

Code Snippet

Tool Used

Manual Review

Recommendation

Change the denominator in the fee tier calculation to use `INIT_HIGH_WATER_MARK` instead of `high_water_mark`:

```
let diff = u256_mul_div(
    x: share_value,
    y: MAX_BASIS_POINTS.into(),
-   denominator: high_water_mark,
+   denominator: INIT_HIGH_WATER_MARK,
    rounding: Rounding::Floor,
) - MAX_BASIS_POINTS.into();
```

This ensures that fee tier selection is based on total performance from inception, while the actual fee amount calculation continues to use `share_value - high_water_mark` to only charge fees on gains since the last collection.

Discussion

MohammadNassar1

Thank you for the suggestion.

I'd like to clarify some points:

1. The `high_water_mark` used in tier selection is not updated on every fee collection, as suggested. It is only updated when the current per-share value exceeds the previous `high_water_mark`.
2. The initial value, `INIT_HIGH_WATER_MARK`, represents the starting per-share price before any deposits are made.

Our current implementation ensures fees are only charged on new yields, avoiding double charging. Also, tier selection is based on relative performance gains since the last all time high, not since the last fee collection. The system can hit higher fee tiers, for example, when performance spikes extremely in a round.

0xEVom

1. "on every fee collection" -> every time performance fees are collected, which is when the share value exceeds the high water mark
2. tier selection should be based on relative performance from baseline. the selected tier would then be applied indeed only to the new yield. the current method will only select a higher tier if the vault's performance is manually updated with a significant value increase, and it will take lower fees if the vault's performance is instead updated more frequently. this does not make sense and is likely not intended

Issue L-1: Incorrect balance calculation returns wrong available withdrawal amount

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/48

Summary

The `get_available_assets_for_withdrawal()` function incorrectly calculates the available withdrawal balance by checking the multisig's balance of strategy shares instead of the underlying asset balance.

Vulnerability Detail

In manual_strategy::get_available_assets_for_withdrawal(), the function attempts to determine how many assets are available for withdrawal by taking the minimum of the deposited amount and the multisig's balance.

However, `self.erc20.balance_of(self.get_multi_sig())` accesses the ERC20 interface of the current contract (the strategy contract itself), which means `balance_of()` returns the multisig's balance of ERC4626 shares of this strategy contract, not the underlying asset balance.

According to the project's design, assets are transferred to the multisig while the vault holds the shares. This is evident in the `after_deposit()` hook where assets are transferred to the multisig:

Since only the underlying assets are transferred to the multisig (not the strategy shares), the multisig's balance of strategy shares will likely always be 0, making the function return 0 instead of the actual available withdrawal amount.

Impact

The function will consistently return incorrect values for available withdrawal assets, typically returning 0 when assets are actually available in the multisig. While this doesn't directly affect the withdrawal mechanism itself, it can lead to poor user experience or integration issues.

Code Snippet

```
File: manual_strategy.cairo
170:         fn get_available_assets_for_withdrawal(self: @ContractState) -> u256 {
171:             let deposited_amount = self.get_deposited_amount();
172:             let balance = self.erc20.balance_of(self.get_multi_sig());
```

```
173:             min(deposited_amount, balance)
174:         }
```

Tool Used

Manual Review

Recommendation

Fix the balance calculation to check the multisig's balance of the underlying asset instead of strategy shares:

```
fn get_available_assets_for_withdrawal(self: @ContractState) -> u256 {
    let deposited_amount = self.get_deposited_amount();
-   let balance = self.erc20.balance_of(self.get_multi_sig());
+   let asset_dispatcher = IERC20Dispatcher { contract_address:
 ↪  self.erc4626.asset() };
+   let balance = asset_dispatcher.balance_of(self.get_multi_sig());
    min(deposited_amount, balance)
}
```

Issue L-2: Missing event emission for manual amount updates

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/49

Summary

The manual_strategy::update_amount() function lacks event emission when manually adjusting the deposited amount, reducing transparency and monitoring capabilities for critical state changes.

Vulnerability Detail

The `update_amount()` function allows the app governor to manually adjust the `deposited_amount` state variable independently of actual deposit or withdrawal operations. This function performs critical state modifications but fails to emit any events to log these changes.

Unlike the normal deposit and withdrawal flows that emit `AssetsForwareded` events through the ERC4626 hooks, manual adjustments through `update_amount()` occur silently without any event logging.

The lack of event emission is particularly concerning given that this function can update amounts separately from actual asset movements.

Impact

Off-chain systems and users cannot easily track manual adjustments to the deposited amount, reducing transparency and making it harder to monitor all state changes in the contract.

Code Snippet

Tool Used

Manual Review

Recommendation

Add an event emission to log manual adjustments, such as:

Issue L-3: bridge does not use PausableComponent

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/50

Vulnerability Detail

`bridge` introduces `PausableComponent`, which is not currently used.

Impact

Except for `redeem()` which can be paused by `is_withdrawals_enabled` none of the other functions can be paused. For example, `mint()`.

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/bridge/src/bridge.cairo#L43

Recommendation

`mint()` add pause mechanism.

Discussion

bin2chen66

Fixed remove PausableComponent

Issue L-4: include own prefix in the signature to avoid conflicts.

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/52

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

consortium's current signatures all use the `payload()` prefix, which may lead to conflicts in subsequent extensions.

Vulnerability Detail

`set_next_validator_set()` using prefix `payload`: NEW_VALSET = `bytes4(keccak256("payload(uint256,bytes[],uint256[], uint256,uint256[],")). uint256,uint256)"))`

`bridge.mint()` using prefix `payload`: DEPOSIT_BTC_ACTION= `bytes4(keccak256("payload( bytes32,bytes32,uint64,bytes32,uint32)"))`

in order to extend it and avoiding possible conflicts with subsequent additions. suggest to change it to its own prefix

Also, would consider add DOMAIN_SEPARATOR to hash, similar ERC20Permit::permit() / snip-12

Impact

Subsequent addition of signatures may lead to conflicts

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/consortium/src/utils.cairo#L58

Recommendation

1. use own prefix NEW_VALSET = `bytes4(keccak256("set_next_validator(uint256,bytes[],uint256[],uint256,uint256)")` DEPOSIT_BTC_ACTION = `bytes4(keccak256("deposit(bytes32,bytes32,uint64,bytes32,uint32)"))`
2. add snip-12

Discussion

MohammadNassar1

Thank you for the suggestion! These hash values, the NEW_VALSET and DEPOSIT_BTC_ACTION, are used in the multisig, so we're unable to change it at this stage.

555-andrew

We are aware about it. And gradually changing prefixes with newer versions of smart-contract. Eventually we will remove `payload` prefix completely, but not in this version.

Issue L-5: mint_limit ignore fee, user can deposit more than deposit_limit

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/53

Summary

The `mint_limit()` calculation does not take into account the effect of the fee, and the return value is higher.

Vulnerability Detail

We use `mint_limit()` to limit the maximum number of mints.

```
fn mint_limit(
    self: @ERC4626ComponentState<ContractState>, receiver: ContractAddress,
) -> Option<u256> {
    self
        .deposit_limit(receiver)
        .map(
            |asset_limit| self
                ._convert_to_shares(assets: asset_limit, rounding: Rounding::Floor),
        )
}
```

The above code, does not take into account the effect of the fee

Example: deposit_limit = 200 total_assets = 150 exchange rate = 1:1 fees = 0.04

mint_limit = 200 - 150 = 50

After the user executes mint(50), the user pays assets = 50 * 1 / (1 - 0.04) = 52.08

After that total_assets = 150 + 52.08 = 202.08

This will be greater than the deposit_limit

Impact

allow users to mint slightly more shares through mint() than through deposit().

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/vault/src/vault.cairo#L249

Recommendation

Similar to `adjust_deposit()`, reduce `fee_assets`

Discussion

bin2chen66

Fixed mint_limit = convert_to_shares(deposit_limit) * (1 - fee)

Issue L-6: take_fees should keep DECIMALS OFFSET

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/56

Summary

`take_fees()` doesn't keep `DECIMALS OFFSET`, which affects the `high_water_mark` judgment.

Vulnerability Detail

The code for `take_fees()` is as follows.

`fee_in_shares` need keep `DECIMALS_OFFSET (1e9:1)` , otherwise it affects `accrued_performance_fee ()` when check the `share_value > high_water_mark`

high_water_mark = INIT_HIGH_WATER_MARK = 1e9

Especially when `supply.is_zero()`

Impact

when `accrued_performance_fee()` could not be charged properly.

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/vault/src/vault.cairo#L654

Recommendation

```
                let fee_in_shares = if supply.is_zero() {
-                   total_fee
+                   total_assets *
 ↪  10_u256.pow(ImmutableConfigImpl::DECIMALS_OFFSET.into())
                } else {
-                   u256_mul_div(
-                       x: total_fee,
-                       y: supply,
-                       denominator: total_assets - total_fee,
-                       rounding: Rounding::Floor,
-                   )
+                  u256_mul_div(
+                           x: total_fee,
```

```
+                            y: supply +
 ↪  10_u256.pow(ImmutableConfigImpl::DECIMALS_OFFSET.into()),
+                            denominator: total_assets - total_fee + 1,
+                            rounding: Rounding::Floor,
+                        )
                 };
```

Discussion

bin2chen66

Fixed in https://github.com/starkware-libs/sn-lbtc/pull/348

but the last commit Hash: 75bd81ccb874bda58528f5e64880560b24ebbc05 is not included This PR, has it been removed?

@MohammadNassar1 Could you please confirm this?

bin2chen66

Fixed in 0eaa3bcf17c60033bf6eb759ffd47a4be2656032

use

`denominator: total_assets - total_fee_in_assets` Will not be zero, because`total_fee_in_assets < total_assets`

Issue L-7: stake_and_bake() may be frontrun, resulting in failure

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/57

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

when `stake_and_bake()`, `bridge.mint()` may have been frontrun, resulting in failure

Vulnerability Detail

The code for `stake_and_bake()` is as follows.

bridge.mint() can be executed by anyone, so there is a malicious frontrun here that will cause a failure `errors::PAYLOAD_ALREADY_USED`.

Impact

if the recipient is a contract, it will only get lbtc instead of vault, which might be a bit unintended. recipient logic may not have a way to handle lbtc

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/stake_and_bake/src/stake_and_bake.cairo#L122

Recommendation

add a judgment `bridge.is_payload_used() == true` then does not execute `bridge.mint()`

Discussion

MohammadNassar1

We will not modify the code. In this scenario, the `stake_and_bake` call will fail, and the user will need to deposit tokens by calling the `deposit` on the vault contract.

555-andrew

We are aware about this risk and consider it acceptable.

Issue L-8: stake_and_bake() may pay a higher fee than expected

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/58

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`stake_and_bake()` code as follows.

the self.fee can be modified by the operator and is not time-locked.

So when the transaction is executed, the fee charged may have changed and exceeded the user's expectations.

Impact

pay a higher fee than expected

Code Snippet

https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/blob/6af132981cc2d19fdc3281208020fa0783081404/sn-lbtc/packages/stake_and_bake/src/stake_and_bake.cairo#L119

Recommendation

```
        fn stake_and_bake(
            ref self: ContractState,
            to_chain: u256,
            recipient: ContractAddress,
            amount: u256,
            tx_id: u256,
            vout: u32,
            proof: Span<Signature>,
            deadline: u64,
            signature: Span<felt252>,
+           max_fee: u256,
        ) {
...
            let fee = self.get_stake_and_bake_fee();
            assert(amount > fee, ZERO_DEPOSIT_AMOUNT);
+           assert(fee <= max_fee, INVALID_FEE);
```

Discussion

MohammadNassar1

We won't change the code. Since the fee can be updated only by the operator.

555-andrew

We are aware about this, but consider it acceptable since only account with certain role assigned (= account we control) can call this function.

Issue L-9: Incorrect pending fee calculation in max withdraw leads to underestimated withdrawal limits

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/59

Summary

The vault::withdraw_limit() function incorrectly subtracts the entire pending protocol fee from individual users' withdrawal limits instead of only deducting their proportional share, causing erc4626::max_withdraw() to return severely underestimated values.

Vulnerability Detail

In the `withdraw_limit()` function, the calculation of maximum withdrawable assets incorrectly handles pending protocol fees. The function calculates the total pending fees using `vault.accrued_protocol_fee()` and then subtracts this entire amount from the user's withdrawal limit.

When protocol fees are actually collected via `_take_fees()`, new shares are minted to the fee recipient, which dilutes all shareholders proportionally. However, the current `withdraw_limit()` implementation treats the fees as if they should be borne entirely by the user making the withdrawal query.

Consider a vault with 100,000 total assets where:

- User A owns 1% of shares (1,000 assets worth)
- Vault has accrued 1,000 in pending protocol fees (1% of total assets)

The current calculation would subtract the full 1,000 pending fees from User A's withdrawal limit, potentially resulting in zero withdrawable amount. However, User A should only bear 1% of the pending fees (10 assets), allowing them to withdraw approximately 990 assets.

Impact

While `_take_fees()` in `vault::withdraw()` prevents `pending_fees` from being nonzero when reached through `erc4626::withdraw()`, the calculation remains fundamentally incorrect and causes `max_withdraw()` to return inaccurate values.

Specifically, users will receive underestimated withdrawal limits. This breaks external integrations and user interfaces that rely on accurate withdrawal limit calculations.

Code Snippet

Tool Used

Manual Review

Recommendation

Perform the same computation as in `_take_fees()` and take into account the shares to be minted when converting the user's shares to assets before deducting the withdrawal fee. The calculation should mirror the dilution logic used in fee collection.

Issue L-10: Pause mechanism has unsafe dependency on fee calculation logic

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/61

Summary

The `pause()` function calls `_take_fees()` before pausing, creating a potential single point of failure where fee calculation issues could prevent emergency pausing.

Vulnerability Detail

The vault::pause() function unconditionally calls _take_fees() before executing the actual pause mechanism. This creates an architectural dependency where the critical safety mechanism relies on the successful execution of complex fee calculation logic.

The `_take_fees()` function performs multiple operations including `accrued_protocol_fee()` and `accrued_performance_fee()` calculations, external calls to strategy contracts through `total_assets()`, and various `u256_mul_div` operations. While arithmetic overflows are not realistic with expected value ranges, the function could potentially fail due to misconfigurations causing `u256_mul_div` reverts or external dependency failures when calling the strategy contract.

The design creates an unnecessary coupling between fee collection (a business logic operation) and pausing (a critical safety mechanism). This is particularly concerning because pausing is most likely to be needed during emergency situations, market stress, or potential exploits - precisely the conditions where external dependencies and complex calculations are most likely to behave unexpectedly.

Impact

If fee calculation logic fails unexpectedly, administrators lose the ability to pause the contract during emergencies, forcing reliance on the upgrade mechanism as the only available emergency response.

Code Snippet

Tool Used

Manual Review

Recommendation

Add a `force` parameter to the `pause()` function that allows administrators to bypass fee collection during emergencies:

This maintains the current behavior by default while providing a fail-safe mechanism for emergency situations.

Issue L-11: Non-standard ERC4626 integration prevents vault from working with generic strategies

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/62

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The vault's `calculate_total_assets()` implementation and deposit/withdrawal flows are tightly coupled to the manual strategy's specific behaviour, preventing compatibility with standard ERC4626 strategies.

Vulnerability Detail

The vault's TotalAssetCalcTraitImpl implementation incorrectly calculates total assets by directly calling the strategy's `total_assets()` method instead of using the standard ERC4626 pattern. In a typical ERC4626 vault-strategy relationship, the vault holds shares in the strategy and should calculate its total assets by converting those shares to underlying assets using `convert_to_assets(balance_of(vault))`.

The current implementation works with the manual strategy because it simply returns the `deposited_amount`, but this approach breaks the ERC4626 standard where strategies are expected to issue shares to depositors. When the vault deposits assets into a standard ERC4626 strategy, it receives shares in return, but the current `calculate_total_assets()` implementation ignores these shares entirely.

Additionally, the vault's deposit and withdrawal flows lack proper validation against strategy limits. The `after_deposit()` hook deposits assets without checking the strategy's `max_deposit()` limit, and the withdrawal flow assumes unlimited withdrawal availability.

Impact

If the vault is upgraded to work with standard ERC4626 strategies in the future, the `calculate_total_assets()` function will return incorrect values, leading to wrong share price calculations and potential loss of funds.

Code Snippet

Tool Used

Manual Review

Recommendation

Implement the standard ERC4626 pattern for calculating total assets and add proper limit checks:

```
fn calculate_total_assets(self: @ERC4626ComponentState<ContractState>) -> u256 {
    let vault = self.get_contract();
    let strategy_address = vault.strategy_address.read();
    let strategy_dispatcher = IERC4626Dispatcher { contract_address:
    ↪  strategy_address };

    let vault_shares = strategy_dispatcher.balance_of(get_contract_address());
    strategy_dispatcher.convert_to_assets(vault_shares)
}
```

Additionally, add validation in the deposit flow to check `strategy_dispatcher.max_deposit(get_contract_address())` before attempting deposits, and implement proper withdrawal limit validation using the strategy's `max_withdraw()` function.

Discussion

MohammadNassar1

Thanks for the suggestion. Currently, our code supports a single strategy, so we don't need to work with generic strategies.

Issue L-12: Function processes unsupported output types instead of reverting

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/65

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The get_dust_limit_for_output() function continues execution when encountering `OutputType::UNSUPPORTED` instead of reverting, which goes against defensive programming principles.

Vulnerability Detail

In the `get_dust_limit_for_output()` function, when the `output_type` parameter is `OutputType::UNSUPPORTED`, the function proceeds with the calculation by adding `NON_WITNESS_INPUT_SIZE` to the `spend_cost`. This behaviour contradicts the explicit design intent of the `UNSUPPORTED` variant, which should indicate that the output type cannot be processed. From a defensive programming perspective, continuing execution with explicitly unsupported inputs weakens the system's robustness.

The function currently treats unsupported outputs identically to non-witness outputs by applying the same spending cost assumptions, which may not be appropriate for truly unknown or unsupported script types.

Impact

Processing unsupported output types could lead to incorrect dust limit calculations or undefined behaviour.

Code Snippet

Tool Used

Manual Review

Recommendation

The function should revert when encountering `OutputType::UNSUPPORTED`.

Discussion

555-andrew

This part was updated in PR https://github.com/starkware-libs/sn-lbtc/pull/299/files to match implementation of EVM version. And since the caller panics in case of `UNSUPPORTED` script type, this change should be sufficient.

Issue L-13: Small deposits may not meet redemption requirements

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/66

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The bridge contract accepts deposits of any non-zero amount but enforces minimum thresholds during redemption, potentially trapping small deposits that cannot meet withdrawal requirements.

Vulnerability Detail

The bridge::mint() function only validates that the deposit amount is non-zero, whilst the redeem() function enforces additional constraints including burn commission fees and Bitcoin dust limits. This asymmetry means users may be able to successfully deposit amounts that will later fail redemption due to insufficient funds to cover fees or meet dust thresholds.

Impact

Users may deposit small amounts that cannot be withdrawn due to redemption requirements, though this primarily affects dust-level amounts and depends on the minimum amount that can be deposited through the Bitcoin side of the bridge.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider adding a minimum deposit amount validation in the `mint()` function to align with redemption requirements, or document the expected minimum deposit amounts.

Discussion

555-andrew

We are aware about this. Minimum deposit amount is being enforced on Ledger so no reason to enforce it on the smart-contract side. Also if have information about minimum amount to deposit being communicated to the user in UI. EVM version of our smart-contracts does it the same way.

Issue L-14: Fee limits allow economically prohibitive values

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/67

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Multiple fee-setting functions across the bridge and vault contracts either lack upper bounds entirely or permit economically prohibitive values that could render the protocols unusable.

Vulnerability Detail

The protocol implements various fee mechanisms with inadequate upper bound protections, creating scenarios where fees could be set to levels that make the services economically unfeasible.

The bridge contract's change_dust_fee_rate() function has no upper limit, allowing arbitrarily high dust fee rates. Since withdrawals must satisfy `assert_with_byte_array(is_above_dust, errors::amount_below_dust_limit(:dust_limit))`, excessively high dust rates would make all withdrawals fail by creating impossibly high dust limits.

The vault contract allows deposit, withdrawal, protocol, and performance fees up to MAX_BASIS_POINTS (100%). Setting fees at or near 100% would consume entire transaction values, making protocol interaction economically irrational.

Impact

A compromised app governor could set fees to disable the protocols: infinite dust rates would block all bridge withdrawals, whilst 100% vault fees would consume all user funds during transactions.

Code Snippet

Tool Used

Manual Review

Recommendation

Implement reasonable upper bounds for all fee parameters based on typical operational costs and market standards.

Discussion

555-andrew

We are aware about this approach to the limits. It is intentional. EVM version of our smart-contracts does it the same way.

Issue L-15: Dust limit check is overly restrictive compared to Bitcoin Core standards

Source: https://github.com/sherlock-audit/2025-06-starkware-lombard-integration/issues/68

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The bridge contract uses a strict greater-than comparison when checking dust limits, which is more restrictive than Bitcoin Core's definition and may reject valid transactions that are exactly at the dust threshold.

Vulnerability Detail

The _calculate_fee_and_dust_limit() function determines whether a withdrawal amount is above the dust limit using a strict comparison. The function calculates the `dust_limit` using `get_dust_limit_for_output()` and then checks if `amount_after_fee > dust_limit.into()`. However, according to Bitcoin Core documentation, dust is defined for a typical spendable segwit P2WPKH transaction output as being *less than* the calculated limit. This means that amounts equal to the dust limit should be considered valid and not classified as dust.

Impact

Users attempting to withdraw amounts that are exactly equal to the dust limit will have their transactions rejected in an edge case scenario. This creates a minor inconsistency with Bitcoin Core's dust definition.

Code Snippet

```
let is_above_dust = amount_after_fee > dust_limit.into();
```

Tool Used

Manual Review

Recommendation

Change the comparison operator from strict greater-than to greater-than-or-equal-to align with Bitcoin Core's dust definition.

Discussion

555-andrew

We are aware about this restriction. It is intentional. EVM version of our smart-contracts does it the same way.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.